



Universidad de Carabobo
Facultad Experimental de Ciencias y Tecnología
Departamento de Computación
Fundamentos de programación



Cod. Asignatura: TAO207

Proyecto (valor: 20%)

Dungeon Crawler en consola

Normas de Entrega:

La entrega del proyecto se realizará durante la última semana del semestre, en la fecha y lugar que serán anunciados previamente por el profesor.

1. Contenido

Se deberá entregar un archivo comprimido en formato .zip que contenga:

- Un único archivo fuente con extensión .c, que incluya todo el desarrollo del proyecto.
- Un informe cuyo contenido se describe en las páginas subsiguientes.
- Los archivos de entrada correspondientes.
- No deben incluirse archivos de salida ni ejecutables generados por el compilador (gcc).

2. Codificación

- El nombre del archivo .c y el de los archivos de entrada debe coincidir exactamente con lo indicado en el enunciado del problema.
- El archivo fuente debe contener en su cabecera, mediante comentario de bloque, la siguiente información: o Fecha o Nombre y Apellido o Cédula o Número de Sección.

3. Defensa oral (Obligatoria)

- Cada equipo deberá realizar una defensa oral del proyecto, en la fecha y horario establecidos por el profesor.
- Durante la defensa, los integrantes deberán explicar el funcionamiento del programa, las decisiones de diseño tomadas, y responder preguntas sobre su implementación.

4. Informe

Se debe entregar un informe en formato digital, incluido en el archivo comprimido, que contenga los siguientes apartados:

1. Portada

- Título del proyecto
- Nombres y apellidos de los integrantes
- Cédulas de identidad
- Sección
- Fecha de entrega

2. Descripción del problema

- Explicación general del enunciado y objetivos
- Contexto y propósito del software desarrollado

3. Análisis de la solución

- Cómo se organizan y almacenan los datos
- Justificación de las decisiones tomadas

4. Estructuras de datos

- Definición de los structs utilizados
- Arreglos u otras estructuras aplicadas
- Diagrama o representación opcional

5. Algoritmos implementados

- Explicación en lenguaje pseudoformal
- Puede usarse pseudocódigo, texto estructurado o diagramas

6. Pruebas realizadas

- Descripción de los casos de prueba
- Breve resumen del comportamiento observado

7. Conclusiones

- Reflexión sobre la experiencia de desarrollo
- Dificultades encontradas y soluciones
- Posibles mejoras

El informe debe estar redactado en letra Arial o Calibri (Word) o Roman (LaTeX), tamaño 12 puntos, con márgenes de 3 cm a la izquierda y 2 cm en los demás bordes.

5. Importante

- El proyecto debe ser realizado en equipos de 2 (dos) estudiantes de la misma sección de laboratorio, sin excepción. No se aceptarán cambios de equipos una vez conformados.
- El programa debe desarrollarse exclusivamente en lenguaje ANSI C, utilizando las librerías estándar: `stdio.h`, `ctype.h`, `string.h`, `stdlib.h`, `assert.h`, `time.h` y `math.h`.
- La solución debe aplicar únicamente los contenidos vistos en clase, tales como: Condicionales, Ciclos, Acciones nominadas, Archivos, Arreglos y Registros.
- Es obligatorio el uso de acciones nominadas (subrutinas con nombres significativos) y parámetros en el código.
- El código debe estar bien documentado y estructurado, con comentarios, indentación clara y uso de nombres significativos.

Considere seguir algún formato de documentación compatible con herramientas ampliamente utilizadas tales como doxygen (<https://www.doxygen.nl/>), o en su lugar, considere utilizar sphynx (<https://www.sphinx-doc.org/en/master/>).

- El informe será evaluado también en cuanto a su redacción y ortografía.
- Los proyectos entregados después de la fecha tope tendrán una penalización de 5 puntos por cada día de retraso.
- Los criterios de evaluación serán publicados oportunamente por el profesor. No seguir las indicaciones antes mencionadas repercutirá en la nota final de su entrega.

Enunciado

1. Contexto y Rol del Desarrollador

Se ha asignado al programador el desarrollo del juego en consola denominado "El Descenso al Núcleo". El desarrollador actuará como ingeniero de software encargado de modelar las entidades, la gestión de memoria y el flujo de ejecución del sistema. El objetivo es garantizar que el código sea eficiente, modular y cumpla estrictamente con las especificaciones técnicas requeridas para simular una expedición a una mazmorra subterránea.

Utilizando la consola el jugador debe poder avanzar e interactuar.

2. Restricciones Técnicas

Control de Aleatoriedad (Semilla): Dado que el sistema requiere la generación de eventos pseudoaleatorios (cálculo de daño, aparición de objetos), el programa debe inicializar la aleatoriedad utilizando una semilla provista en el archivo de configuración. Esta semilla se pasará a la función `srand()` para garantizar que los resultados de una partida puedan ser replicados exactamente para fines de auditoría y evaluación.

3. Entorno y Navegación

El juego debe funcionar a través de un sistema de menús por consola

El menú de inicio debe tener las siguientes opciones:

- empezar nueva partida
- cargar partida (si existe el archivo `partida_guardata.txt`)
- highscores, debe cargar el archivo de `highscore.txt` y mostrar por consola las últimas 5 victorias
- salir

Al estar fuera de combate el jugador tendrá un menú por consola donde podrá:

- avanzar al siguiente cuarto o piso, si se acabaron los pisos el jugador avanza al núcleo y termina el juego. Avanzar al siguiente cuarto o piso **siempre empieza un combate** contra un enemigo random dependiendo del cuarto y del tipo del enemigo.
- revisar su mochila, donde puede seleccionar un objeto para utilizarlo o soltarlo, o puede ir hacia atrás en el menú
- guardar y salir de partida

Al estar en combate el jugador tendrá un menú por consola donde podrá:

- Atacar al enemigo
- Revisar su mochila, donde puede seleccionar un objeto para utilizarlo o soltarlo, o puede ir hacia atrás en el menú

- Tratar de huir, tratar de huir debe tener una probabilidad de éxito calculada usando el ataque del enemigo y la defensa del jugador, el programador puede diseñar su propia función de éxito de escape, huir exitosamente lleva al jugador a estar fuera de combate.
- Las 3 acciones mencionadas consumen el turno del jugador y luego el enemigo debe atacar al jugador.
- El combate continuará hasta que el HP del jugador o del enemigo sea menor o igual que 0, o hasta que el jugador logre huir

Al empezar partida el jugador se encontrará fuera de combate en el cuarto 0 piso 1, cuando el jugador avance al cuarto 1 empezará su primer combate

Recorrido Lineal por Cuartos: La progresión de la mazmorra se divide en pisos, y a su vez, cada piso está compuesto por una serie de cuartos. El jugador avanza linealmente de un cuarto a otro; al limpiar todos los cuartos de un piso, desciende automáticamente al siguiente. No existe la posibilidad de retroceder.

Distribución de Cuartos: Todos los pisos comparten una cantidad base de cuartos definida por configuración. Sin embargo, por diseño del sistema, el primer piso siempre tendrá un cuarto menos que la cantidad base, y el último piso tendrá un cuarto extra (donde residirá el encuentro final). Cada cuarto debe tener un enemigo.

Abstracción del Escenario: El escenario no tiene representación espacial física. No existen mapas, obstáculos físicos, paredes ni coordenadas (X, Y). La ubicación del jugador se define exclusivamente por el número de piso y el número de cuarto actual.

4. Condición de Victoria y Derrota

Victoria: El juego culmina exitosamente cuando el jugador supera el último cuarto del último piso de la mazmorra.

Derrota: La simulación finaliza en fracaso de manera inmediata si los Puntos de Vida (HP) del jugador se reducen a cero o menos en cualquier momento.

5. Sistema de Progresión (Experiencia y Niveles)

Además de los atributos de Vida, Ataque y Defensa, el jugador cuenta con un sistema de progresión.

Al derrotar a un enemigo, el jugador obtiene Puntos de Experiencia (XP).

Al alcanzar un umbral determinado de XP, el jugador "sube de nivel", lo que debe incrementar automáticamente sus estadísticas base (HP Máximo, Ataque base y Defensa base). El programador debe definir lógicamente los umbrales de XP requeridos para cada nivel.

6. Sistema de Combate y Recompensas (Loot)

Combate por Turnos: Al encontrar una criatura, el sistema alterna ataques entre el jugador y el enemigo.

Cálculo de Daño: El daño de cada golpe se calcula restando la defensa del receptor al ataque del agresor. Por regla estricta de balance, el daño resultante nunca puede ser inferior a 1 (todo ataque exitoso inflige al menos 1 punto de daño).

Recompensas (Enemy Drops): La única forma de obtener consumibles o mejoras para el inventario es como recompensa tras derrotar a un enemigo.

Para los enemigos de tipo Común, el sistema debe determinar aleatoriamente si suelta un objeto al morir

Garantía de Botín: Si el enemigo derrotado es de tipo Raro (ubicados obligatoriamente al final de cada piso), el sistema debe garantizar la caída de un objeto (probabilidad del 100%). En ambos casos, el objeto entregado se seleccionará aleatoriamente del catálogo cargado en memoria.

7. Gestión de Inventario y Persistencia de Partida

Mochila Limitada: El inventario está restringido a un máximo de N espacios. El desarrollador debe utilizar un arreglo (Array) de estructuras para gestionar esta limitante. El jugador puede consumir objetos de su inventario para alterar sus estadísticas.

Guardado y Carga (Suspensión): El sistema debe ofrecer la opción de suspender la expedición actual. Al hacerlo, se debe volcar el estado completo del juego (atributos, nivel actual, experiencia, piso, cuarto e inventario) en un archivo de texto temporal (ej. partida_guardada.txt). Al reiniciar el software, el sistema debe ser capaz de detectar y leer este archivo para reanudar la partida exactamente donde se dejó.

Al terminar la partida el archivo partida_guardada.txt debe ser eliminado o vaciado.

8. Archivos de Configuración y Entidades (Input)

El sistema debe inicializarse leyendo tres archivos de texto.

El programador debe agregar propiedades nuevas a los archivos o editar las ya existentes para distintos casos de prueba o balance del juego dependiendo de su propio criterio.

1. Archivo de Configuración (config.txt): Contiene los parámetros iniciales. El formato esperado es:

- Semilla
- Número total de pisos
- Cantidad base de cuartos por piso
- Estadísticas iniciales del jugador: Puntos de Vida, Ataque base, Defensa base, tamaño del inventario.

```
42
10
5
100 15 5 5
```

2. **Archivo de Bestiario (bestiario.txt):** Catálogo de enemigos que el programa debe cargar en memoria usando arreglos de estructuras.

Formato: [ID_Enemigo] [Nombre_Enemigo] [Vida_Base] [Daño_Base] [Defensa_Base] [Tipo]

```
1 Rata_Ruinas 20 5 0 Comun
2 Automata_Danado 35 3 10 Comun
3 Cazador_Sombras 15 12 2 Comun
4 Guardian_Nucleo 200 25 15 Raro
5 Espectro_Anciano 100 18 10 Raro
```

Regla de Aparición: El programador debe implementar la lógica para que el último cuarto de cualquier piso siempre genere un encuentro con un enemigo de tipo Raro. En los demás cuartos, se seleccionarán enemigos de tipo Comun.

3. **Archivo de Objetos (objetos.txt):** Define el botín posible.

Formato: [ID_Objeto] [Nombre_Objeto] [Fuerza/Valor] [Tipo_Efecto]

```
1 Poción_Menor 25 Cura_Vida
2 Cristal_Fuerza 3 Buff_Daño
3 Placas_Chatarra 2 Buff_Defensa
4 Elixir_Nucleo 100 Cura_Max
5 Poción_Mayor 50 Cura_Vida
```

Regla de Efecto: El programador debe utilizar la propiedad Tipo_Efecto (mediante comparaciones de cadenas) para definir qué bloque de código se ejecuta al consumir el objeto (ej. curar vida, aumentar ataque permanente, etc.).

9. Archivo de Puntuaciones y Estadísticas (Output / highscore)

Independientemente de si la expedición termina en victoria o derrota, el sistema debe generar un reporte final del rendimiento.

Captura de Datos: Al finalizar la partida, el sistema debe detenerse y solicitar por consola que el jugador ingrese su nombre.

Registro Histórico: El programa recopilará los datos en una estructura de estadísticas y los escribirá en un archivo de salida obligatorio llamado highscore.txt.

Modo de Escritura: Este archivo debe conservar los resultados de los jugadores anteriores (debe abrirse en modo de adición o append).

- Formato del Reporte: Cada registro debe encapsular:
- Nombre del Explorador, el nombre del explorador se pide por consola.
- Resultado (Victoria / Derrota).
- Piso Máximo Alcanzado y Cuartos Explorados.
- Total de Enemigos Derrotados y Objetos Consumidos.
- Estadísticas Finales (Nivel, HP final, Ataque y Defensa definitivos).

10. Arquitectura y Diseño Modular

Se penalizará la entrega de soluciones monolíticas donde la lógica resida en la función main. El estudiante deberá abstraer el problema definiendo estructuras coherentes y comunicando los subprogramas mediante el paso de parámetros por valor y referencia (punteros).

La solución debe basarse en la siguiente propuesta de arquitectura:

```
// --- PROTOTIPOS DE FUNCIONES SUGERIDOS ---

// Módulo 1: Inicialización y Archivos Input
void cargarConfiguracion(EstadoJuego *estado);
void cargarBestiario(Entidad bestiario[], int *total_enemigos);
void cargarObjetos(Objeto catalogo[], int *total_objetos);

// Módulo 2: Lógica de Exploración y Navegación
void avanzarCuarto(EstadoJuego *estado, Entidad bestiario[], int
total_enemigos, Objeto catalogo[], int total_objetos);
Entidad seleccionarEnemigo(Entidad bestiario[], int total_enemigos, int
es_cuarto_final);

// Módulo 3: Sistema de Combate y Progresión
void iniciarCombate(EstadoJuego *estado, Entidad *enemigo);
int calcularDano(int ataque_atacante, int defensa_defensor);
void otorgarRecompensa(EstadoJuego *estado, Objeto catalogo[], int
total_objetos, char tipo_enemigo[]);
void verificarSubidaNivel(EstadoJuego *estado);

// Módulo 4: Gestión de Recursos
void usarObjeto(EstadoJuego *estado);

// Módulo 5: Persistencia y Output (Highscore)
void guardarPartida(EstadoJuego estado);
int cargarPartida(EstadoJuego *estado);
void registrarHighscore(EstadoJuego estado, bool victoria);
```